

Student Name: Jobin Bose

Student ID: 24133771

Github link: <https://github.com/Jobin85/MLP-Depth-vs-Width>

MLP Depth vs Width: Does Architecture Shape Matter More Than Size?

Introduction

When you're building a Multilayer Perceptron (MLP), you're often faced with a question: should you add more layers or should you increase the number of neurons in each layer?

You're about to learn the answer through a real-life experiment with a real dataset that you can replicate. You'll also learn how to read the results and when you should consider that MLPs aren't the best option for a particular problem

The Dataset

Let's use the Energy Efficiency dataset developed by Tsanas & Xifara (2012). It's a dataset with 768 simulated buildings with eight features: relative compactness, surface area, wall area, roof area, overall height, orientation, glazing area, and glazing area distribution. The targets are Y1: Heating Load and Y2: Cooling Load.

The dataset is small, so it's easy to train on it, making it a great option for testing architectural decisions. Also, the dataset is a good mix of linear and non-linear data, making it a great option for a fair comparison.

Preprocessing: What You Must Do Before Training

Step 1 — One-Hot Encode Categorical Variables. While the Orientation and GlazingAreaDistribution columns appear to be numeric, they are actually categorical data. Treating them as ordinals, i.e., as a series of numbers, imposes a false order, e.g., Orientation 5 is greater than Orientation 2. This problem is eliminated by using `pd.get_dummies(..., drop_first=True)`.

Step 2 — Standardize the Features. Neural networks are very sensitive to the scale of the input data. A feature ranging from 0 to 800 can dominate the update of the gradients for the weights during training compared to a feature ranging from 0 to 1. We can use the `StandardScaler` to standardize the data, but we must only use the training data to create the scaler, then apply the scaler to the validation and test data separately, as using the test data to create the scaler and then applying the scaler to the training data would allow test data to leak into the training process.

Step 3 — Split the Data into 70, 15, 15 Percentages. The training data, which constitutes 70 percent of the total, is used to teach the model, the validation data, which constitutes 15 percent of the total, is used to help design the model, and the test data, which constitutes 15 percent of the total, is used to test the model only at the very end of the experiment.

Understanding Depth and Width: The Visual Foundation

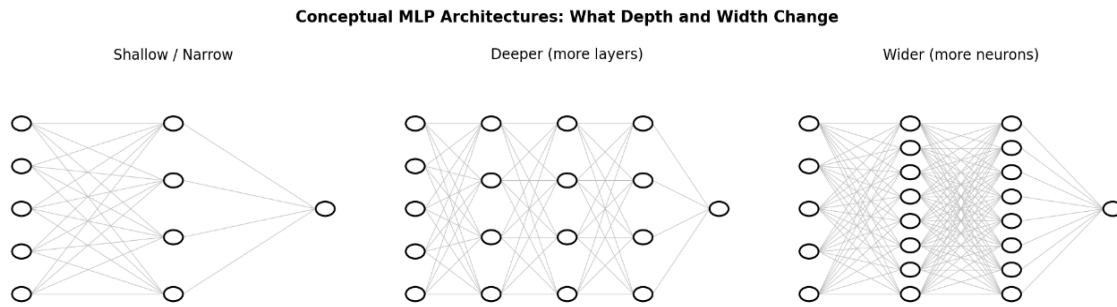


Figure 1: Three MLP configurations. Shallow/narrow (left), deeper with more layers (centre), wider with more neurons per layer (right).

Depth stacks transformations sequentially each layer learns from the abstractions built by the previous one. Theoretically, deep networks can represent certain function families exponentially more efficiently than shallow ones (Raghu et al., 2017). The Universal Approximation Theorem (Hornik et al., 1989) tells us a single layer can approximate any function, but the number of neurons required may be impractical.

Width adds neurons to each layer in parallel, increasing capacity without adding sequential transformations. This is often more stable to train but less efficient for hierarchical patterns.

Glorot initialisation (Glorot & Bengio, 2010) sets initial weights so that activation variance is preserved across layers: $\text{Var}(W) = 2/(\text{fan_in} + \text{fan_out})$. Without this, deep networks can start with exploding or vanishing signals before training even begins.

Experiment 1 — Depth Sweep: Does Going Deeper Help?

What you see in Figure 2:

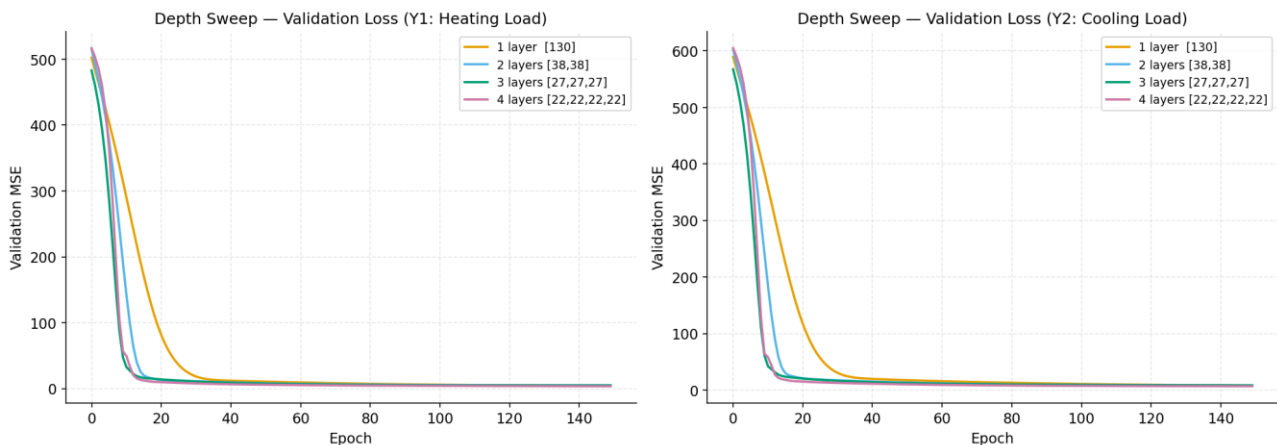


Figure 2: Validation loss curves for four depths on Y1 (left) and Y2 (right). Multi-layer networks converge significantly faster.

Setup: Four architectures were considered with the same number of parameters in each architecture, approximately 1,300. They are 1 layer [130], 2 layers [38,38], 3 layers [27,27,27], and 4 layers [22,22,22,22]. Note that the choice of the number of parameters in the models is important. Increasing the number of parameters in the deeper models could result in performance improvements due to capacity rather than depth.

The 1-layer net converges after about 60-70 epochs. The 2, 3, and 4-layer models achieve low validation loss much earlier in the process, although optimization is still ongoing. This is interesting because the effect of depth not only matters for accuracy but also for the speed of learning. The deeper nets, with the same number of parameters, are learning the problem faster on this data set.

Experiment 2 — Width Sweep: When Does More Become Too Much?

Figure 3 shows two things:

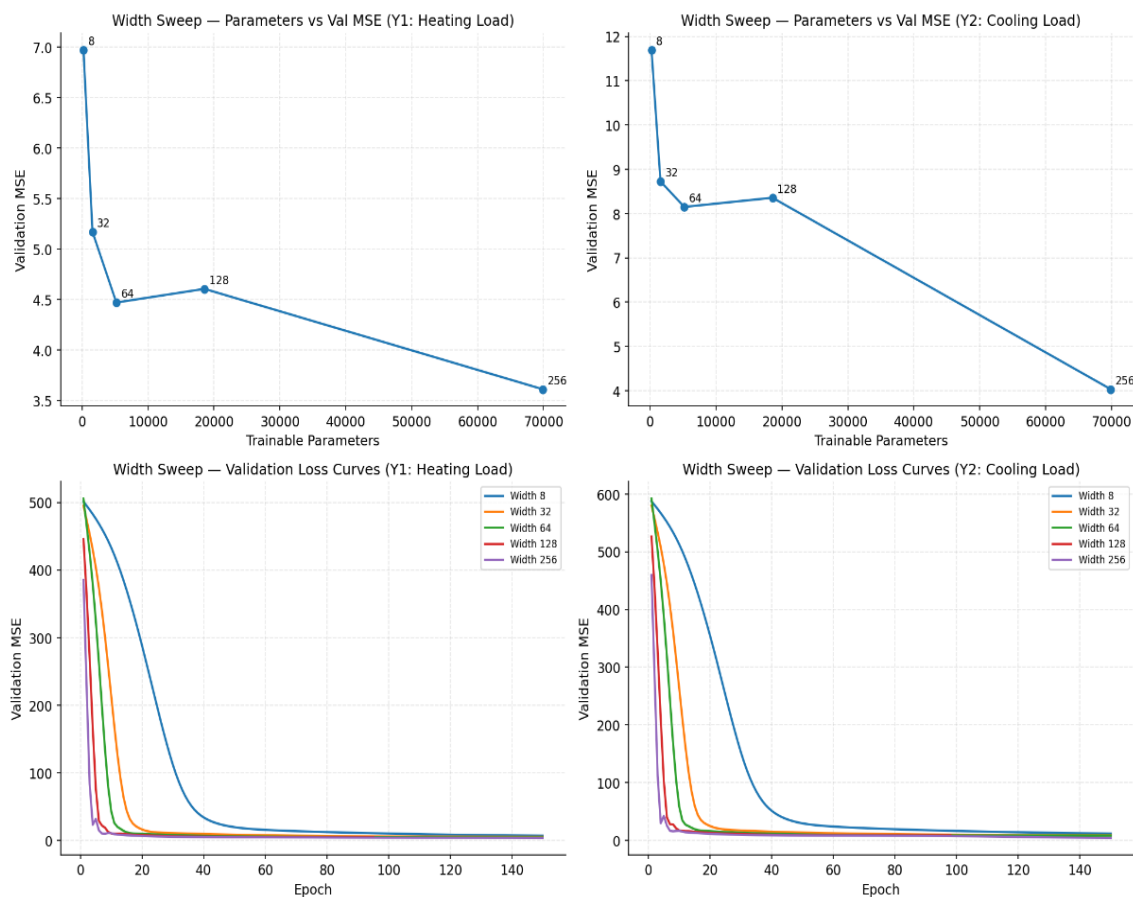


Figure 3 (top): Parameter count vs validation MSE. Figure 3 (bottom): Validation loss curves per width.

Width 8 has significant problems. The greater the width, the faster the convergence. The depth of the network is kept constant at 2. The width varies from 8 to 256.

The width of 8 has an MSE of around 7.0 (Y1), which means that the performance is very poor. The convergence is also very slow. When the width is increased to 32, the MSE drops to 5.2. When the width is increased to 64, the MSE drops to 4.5. At this point, further increase does not yield much. The width of 128 does not yield much compared to width 64

There is an optimal range for width. Widths that are suboptimal have low capacity. Widths that are too wide tend to memorize the noise. For this dataset, the performance improves as the width increases. The best performance is obtained for width 256.

Experiment 3 — Depth $\times$ Width Grid: Both Factors Together

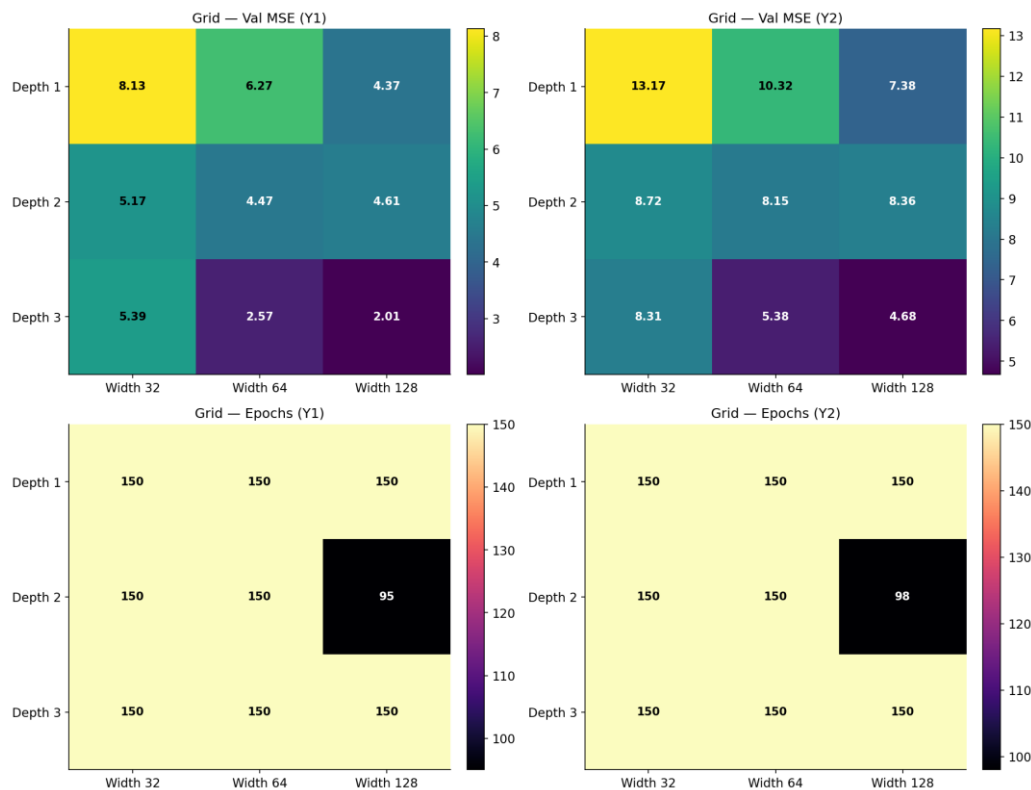


Figure 4: Validation MSE heatmaps (top) and epochs trained (bottom)

Top row (Validation MSE) — Darker (purple) = lower MSE = better. Lighter (yellow) = higher MSE = worse

A 3 $\times$ 3 grid combining depths 1–3 with widths 32, 64, and 128 shows that depth and width interact. Depth 3 with Width 128 gives the best MLP result on both targets, while Depth 1 performs poorly regardless of width.

Bottom row (Epochs trained):

- Almost all show 150, which means early stopping never occurred. Models were still improving. Not ideal for experiments, but it tells you something important: all these configurations need more than 150 epochs to converge completely on this data set.
- One configuration stands out from the rest: Depth 2, Width 128 stopped at 95 (Y1) and 98 (Y2). Why is this one different? The answer is that this is the only network for which early stopping actually occurred. Why is that? That is, it converges to a good solution relatively quickly and then plateaus validation loss does not continue to decrease, patience is exhausted, and training is stopped.
- All the Depth 3 rows show 150, even though they have the best MSE results. That is, they were still learning at the time training stopped they may have been able to achieve even better MSE results.

Overfitting Gap: How Much Are You Memorising?

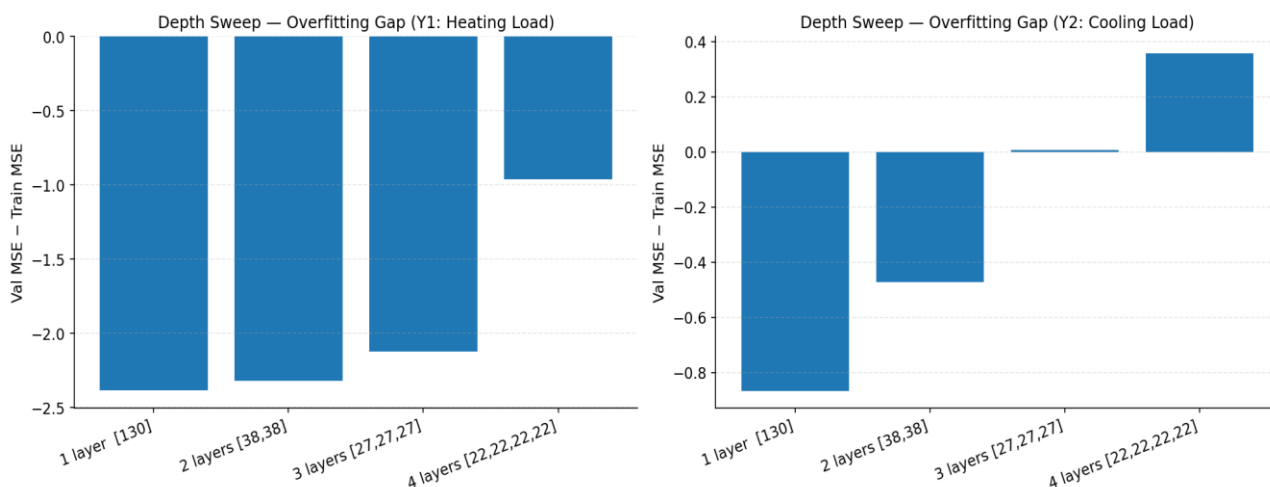


Figure 5: Train–validation MSE gap per architecture.

When you see negative values, it means the validation loss is actually lower than the training loss—something you often find with small datasets that use strong regularization, like early stopping. Looking at the 4-layer network on Y2, there's a positive gap, which shows overfitting is starting to happen. It is because deeper networks tend to overfit more easily on small datasets.

Gradient Norms: Why Depth Can Be Harder to Train

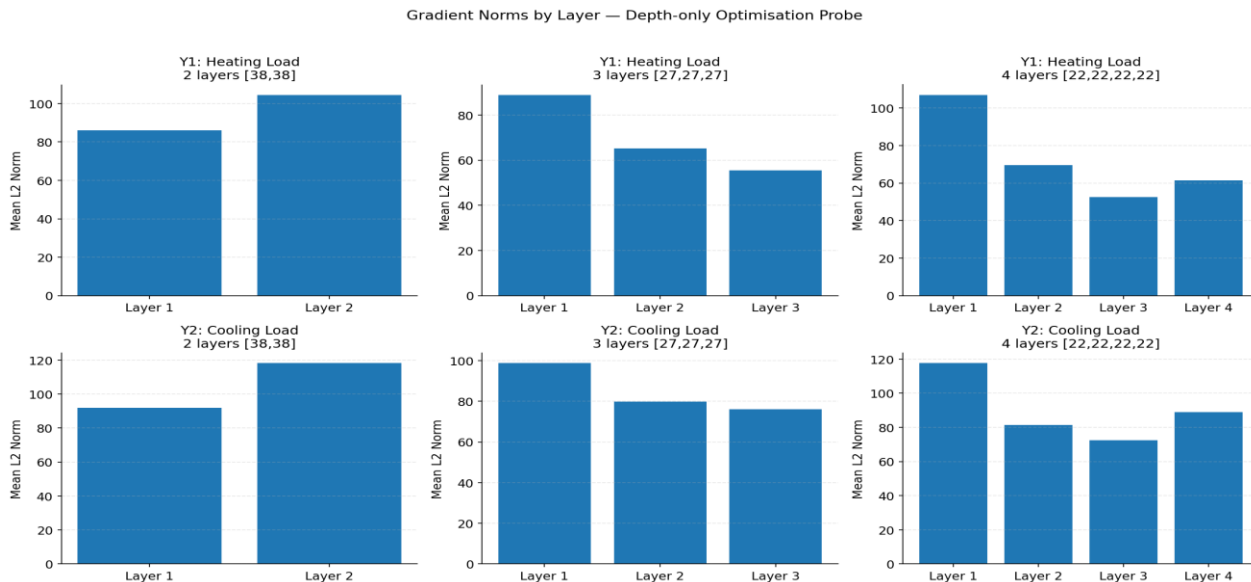


Figure 6: Mean L2 gradient norm per layer.

Take a look at this 4-layer network: the first layer gets a gradient norm of about 105, but by the time you reach layer 3, it drops down to around 53. This kind of gradual weakening of the gradient is a mild version of the vanishing gradient problem — basically, as the network gets deeper, the learning signals for the earlier layers start to fade. Using ReLU activations really helps compared to sigmoid or tanh, which would squash the gradients even more. In practice, if your network is very deep and those early layers aren't learning well, it's a good idea to try techniques like batch normalization, or adding residual connections.

Multi-Seed Robustness: Why One Run Is Not Enough

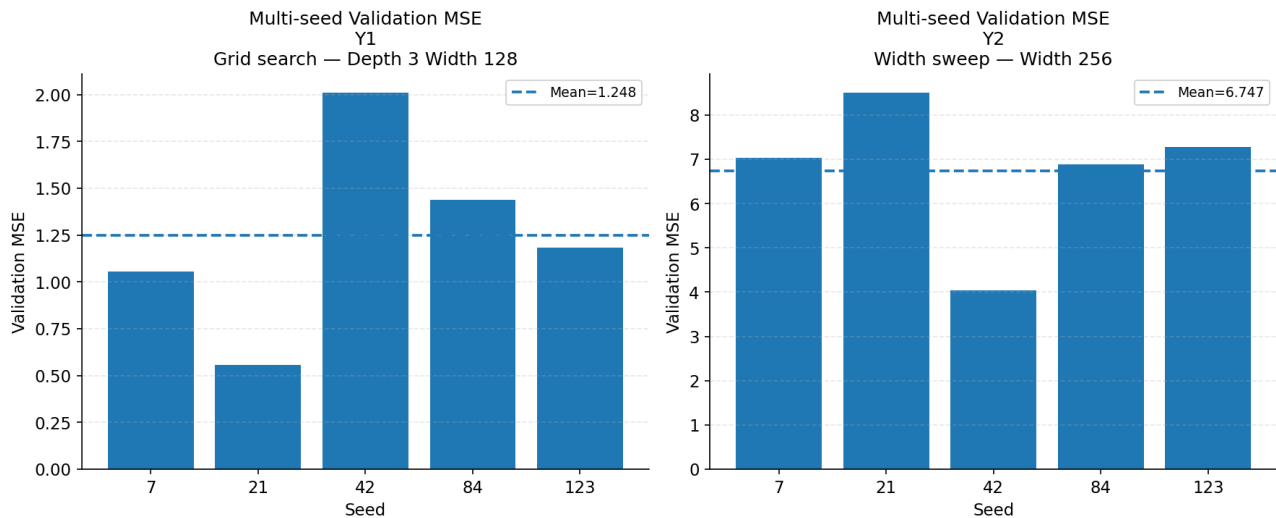


Figure 7: Validation MSE across 5 seeds.

For Y1 (Depth 3, Width 128), the results vary quite a bit depending on the seed—ranging from about 0.55 to 2.0, which is a fourfold difference. With Y2, one seed hits around 4.0, while another jumps up to about 8.7. This just goes to show why you should never rely on a single seed. It's always best to run 3 to 5 seeds and report the average along with the standard deviation. Otherwise, a lucky seed might make a weak architecture seem much better than it really is.

Residual Analysis: Checking Your Final Model

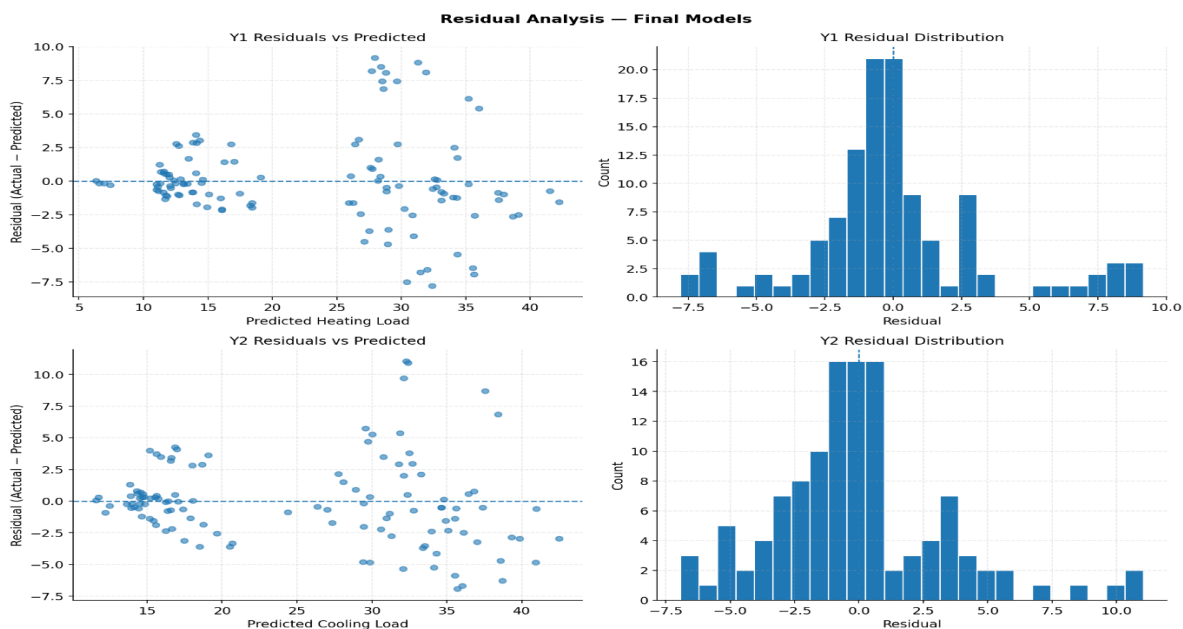


Figure 8: Residuals vs predicted (left) and residual distributions (right) for both targets.

The fact that the residuals cluster around zero shows that the predictions are unbiased, which is good. However, if you look at both graphs, you'll notice a fan-shaped pattern where the residuals spread out more as the predicted values get higher. This suggests the model struggles more with accurately predicting high-energy buildings.

The Most Important Plot: Baseline Comparison

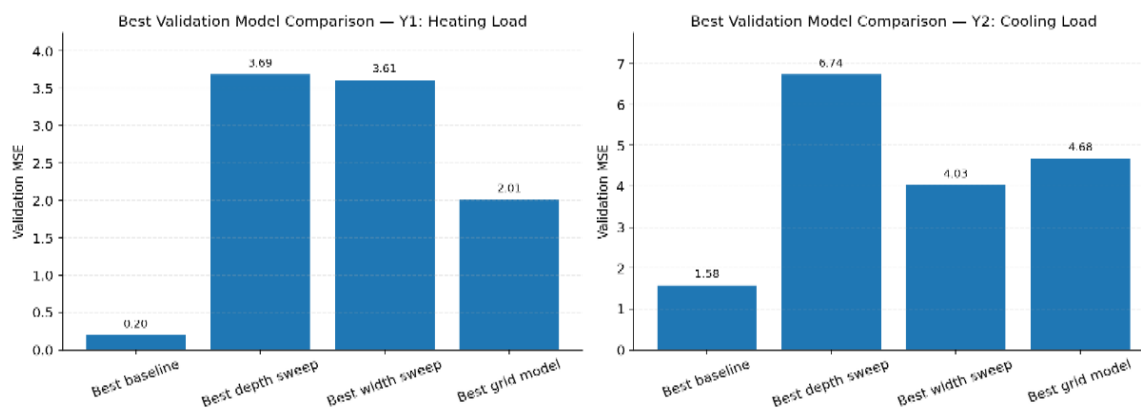


Figure 9: Best validation MSE across all approaches.

To ensure a fair comparison, we used three baseline models: Linear Regression, Ridge Regression, and Random Forest Regression. Figure 9 above shows that Random Forest achieved the lowest validation mean squared error (MSE): 0.20 for Y1 and 1.58 for Y2. This indicates that Random Forest outperforms all the configurations of the MLP models. In fact, the best MLP has an MSE of 2.01 for Y1, which is ten times worse than the Random Forest model's performance on the same variable.

The performance of the Random Forest model is surprising. Neural networks, while not ineffective, often struggle unless the dataset is large or the data is complex and not in a tabular format.

Conclusion

In this tutorial, we have verified that both dimensions are important in the creation of the MLP. Depth helps in the learning dynamics, and width has proven to have diminishing returns on the parameters. The best MLP had both Depth and Width set to 3 and 128 respectively.

The deeper models have also shown higher possibilities of overfitting, and the results have been quite variable with the use of different random seeds.

Most importantly, Random Forest has proven to be the best among all the MLP models on both validation and test data.

For the final test/val scores, on the test data, we have MSE of 10.99 and 11.10 for Y1 - Y2, whereas Random Forest has (0.27 and 3.99). This proves that the best architecture doesn't necessarily mean the best performance, at least on these smaller structured data sets.

Accessibility

All images use the safe color palette as proposed by Wong (2011), which guarantees distinguishability for all kinds of color blindness. The heat maps include numeric annotation to avoid information loss in the absence of color vision. The tutorial uses clear headings to enable efficient navigation for the screen reader. The code in the notebook is combined with markdown to avoid the need for interpreting the code alone. The font size is standardized to at least 11 points for better readability.

References

1. Tsanas, A. and Xifara, A. (2012). Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools. **Energy and Buildings**, 49, pp. 560–567.
2. Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. **Proceedings of AISTATS**, pp. 249–256.
3. Hornik, K., Stinchcombe, M. and White, H. (1989). Multilayer feedforward networks are universal approximators. **Neural Networks**, 2(5), pp. 359–366.
4. Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. **International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems**, 6(2), pp. 107–116.
5. LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. **Nature**, 521, pp. 436–444.
6. Goodfellow, I., Bengio, Y. and Courville, A. (2016). **Deep Learning**. MIT Press. Available at: <https://www.deeplearningbook.org/>
7. Wong, B. (2011). Color blindness. **Nature Methods**, 8, p. 441.